

Infix to Postfix Conversion & Recursion

Q. Discuss the application of Stacks in evaluating mathematical expressions. Explain the process of Infix to Postfix conversion, Postfix evaluation, and how stacks handle Recursion.

> **Definition to Remember**

> **Inf**
no par
also m

1. Operator Precedence (for conversion)

| Operator | Precedence |

| :--- | :

2. Infix to Postfix Conversion Algorithm

1. Scan expression left to right.

2. **Op**

Example: `A \* (B + C)`

| Symbol | Stack | Output |

| :--- | :

3. Postfix Evaluation Algorithm

1. Scan postfix string left to right.

2. **Op**

Example: `5 3 + 2 \*` $(5+3)*2 = 16$

4. Recursion and the Call Stack

- When a function calls itself, the OS uses the **Call Stack** to manage it.

- Eac

```

factorial(3) 3 top (most recent call)

factorial

factorial

```

> **Must-Write Points (for 10 marks)**

> 1. In

> **Quick Recall**

> `Infix

Recurs